

DevOps Homework #1 — Foundations

Level	Beginner
Estimated time	1-2 hours
Format	Short written answers + 2 small hands-on labs
Total	110 points (pass mark 70)

How to submit

1. Answer in any document you like (Google Doc, Markdown, Word) — keep the section and question numbers so it can be graded.
2. For the labs, include the **live URLs** and **screenshots**, and paste terminal output where asked.
3. Put the document in a Git repo named `devops-hw1` and send the repo link. (*You need Git for Section 9 anyway.*)
4. Blank cells in the tables below are for your answers — short phrases are fine, full sentences are not required unless the question asks for them.

Rules: You may Google anything. You may **not** paste an AI-generated answer you can't explain out loud — Section 11 assumes you can defend every line.

Section 1 — DevOps: Why? (8 pts)

1.1 In **2-3 sentences**, describe the problem DevOps was invented to solve. Do not use the words "culture" or "synergy". (3 pts)

1.2 A company deploys once every 4 months. Each release takes a weekend and roughly 1 in 3 releases gets rolled back. List **three concrete problems** this causes for (a) the business and (b) the engineers. (3 pts)

1.3 True or false, and explain in one line each: (2 pts)

#	Statement	T/F	Why
a	DevOps means "developers do operations work too."		
b	If you have a CI/CD pipeline, you are doing DevOps.		
c	DevOps aims to make deployments boring.		
d	DevOps is a job title.		

Section 2 — The DevOps Profession (8 pts)

2.1 Fill in the table. One short phrase per cell. (4 pts)

Role	Main goal	One daily task	One tool
DevOps Engineer			
SRE			
Sysadmin			
Platform Engineer			

2.2 List **five skills** you personally need to work on to become hireable as a junior DevOps engineer. Be specific — "Linux" is not an answer, "writing a systemd unit file from scratch" is. (3 pts)

2.3 Name one thing a DevOps engineer is **not** responsible for, and say who owns it instead. (1 pt)

Section 3 — SRE: SLA / SLO / SLI and Incidents (14 pts)

3.1 Define **SLI**, **SLO**, and **SLA** in one sentence each, then order them from "internal target" to "legal/contractual promise". (3 pts)

3.2 Your API has an SLA of **99.9% monthly uptime**. Calculate the allowed downtime per month (assume a 30-day month). Show the arithmetic. (2 pts)

3.3 Complete the table (30-day month): (2 pts)

Uptime	Allowed downtime / month
99%	
99.9%	
99.99%	

3.4 Explain **error budget** in your own words, and answer: your monthly error budget is already 90% consumed on day 12. A developer wants to ship a risky refactor today. What do you say and why? (3 pts)

3.5 — **Incident scenario**. At 03:12 the checkout page starts returning 500s. You are on call. Write your **first 15 minutes** as a numbered timeline (aim for 5–7 steps). Your answer must include: who you notify, how you decide between *fix forward* and *rollback*, and where you write things down. (3 pts)

3.6 What is a **blameless postmortem**, and name **two** things that must never appear in one. (1 pt)

Section 4 — CSR vs SSR (10 pts)

4.1 Fill in the table with  /  / "depends" plus a few words of justification. (4 pts)

	CSR	SSR
First paint speed on slow phone		
SEO out of the box		
Server CPU cost		
Works with a static host (no server)		

4.2 For each product, choose CSR or SSR and justify in one sentence: (3 pts)

- a) A public news site that lives on Google traffic
- b) An internal admin dashboard behind a login
- c) A marketing landing page with a contact form

4.3 Where does **S SG (static site generation)** fit between CSR and SSR? One short paragraph. (2 pts)

4.4 Open any website, view page source (), and say whether it looks CSR or SSR — and what evidence in the HTML told you. Paste the URL. (1 pt)

Section 5 — Cloudflare & Cloudflare Pages (10 pts)

5.1 Explain what a **CDN** does, using a concrete example of a user in Yerevan requesting an image hosted in Virginia. (2 pts)

5.2 Match the Cloudflare feature to what it does: (3 pts)

Feature	What it does
DNS	
Proxy (orange cloud)	
WAF	
Cloudflare Pages	
Cloudflare Workers	
Cache	

5.3 What is the difference between **orange cloud (proxied)** and **grey cloud (DNS only)**? Give one situation where you'd deliberately choose grey. (2 pts)

5.4 Cloudflare Pages vs a \$5/month VPS running Nginx for a static site: give **two advantages of each**. (2 pts)

5.5 What is a Cloudflare Pages **preview deployment**, and why is it useful in a team? (1 pt)

Section 6 — LAB A: Deploy a website to Cloudflare Pages (14 pts)

Free tier only. No credit card required for this lab.

Steps

1. Create a Git repo (GitHub/GitLab) named `cf-pages-lab`.
2. Add an `index.html` that displays **your name, today's date**, and the text `DevOps HW1`. Add basic CSS. Optionally add a `style.css` — no framework needed.
3. Push to `main`.
4. Create a free Cloudflare account → **Workers & Pages** → **Create** → **Pages** → **Connect to Git**.
5. Select the repo, keep the default build settings (no build command for plain HTML, output directory `/`), and deploy.
6. Wait for the build to go green and open the `*.pages.dev` URL.
7. Change something visible in `index.html`, commit, push. Watch the new deployment appear.
8. Create a branch `feature/color`, change the background colour, push, and open the **preview** URL.

Deliverables

- [] `*.pages.dev` production URL (2 pts)
- [] Repo URL (1 pt)
- [] Screenshot of the Deployments list showing **at least 3** deployments (3 pts)
- [] The **preview** URL from the `feature/color` branch (3 pts)
- [] Paste the build log lines that show the deploy succeeded (2 pts)

Questions

6.1 What triggered deployment #2? Which exact commit? (1 pt) **6.2** Production and preview URLs both work at the same time. Why is that useful, and what is the risk if a preview URL leaks? (2 pts)

Section 7 — LAB B: Buy a domain and bind it to your Pages site (16 pts)

⚠ This lab costs real money — budget **\$1-12/year**. Cheap TLDs: `.xyz`, `.top`, `.site`, `.online`. If you cannot spend money, do the **Alternative** at the end of this section instead — you still get full points.

Part 1 — Purchase (4 pts)

1. Compare the **same domain name** on **three** registrars (e.g. Namecheap, Porkbun, Cloudflare Registrar, GoDaddy).

- Record for each: **first-year price**, **renewal price**, whether **WHOIS privacy** is free, and whether **transfer-out** is free.
- Buy the one you chose.

Registrar	1st year	Renewal	WHOIS privacy	Transfer-out

Chosen registrar + one-sentence reason: _____

7.1 Why is the **renewal** price more important than the first-year price? (1 pt) **7.2** What is **ICANN WHOIS verification**, and what happens if you ignore the confirmation email? (1 pt)

Part 2 — Bind it to Cloudflare Pages (8 pts)

- Add the domain to Cloudflare (**Add a site**), pick the Free plan.
- Copy the **two nameservers** Cloudflare gives you, and set them at your registrar.
- Wait for the zone to become **Active** (minutes to hours).
- In your Pages project → **Custom domains** → **Set up a domain** → add `yourdomain.xyz` and `www.yourdomain.xyz`.
- Confirm HTTPS works and there is no certificate warning.

Deliverables

- [] Screenshot: registrar page showing Cloudflare's nameservers set (2 pts)
- [] Screenshot: Cloudflare zone status **Active** (1 pt)
- [] Screenshot: Cloudflare **DNS records** table for your zone (2 pts)
- [] Working `https://yourdomain.xyz` and `https://www.yourdomain.xyz` (3 pts)

Part 3 — DNS record questions (4 pts)

7.3 Fill in the table — what is each record type *for*? (2 pts)

Type	Purpose	Example value
A		
AAAA		
CNAME		
MX		
TXT		
NS		

7.4 Which record type(s) did Cloudflare create for your apex domain (`yourdomain.xyz`) and which for `www` ? Why can't a plain apex domain normally hold a CNAME — and what does Cloudflare do about it? (1 pt)

7.5 Run these and paste the output: (1 pt)

```
dig yourdomain.xyz +short
dig www.yourdomain.xyz +short
dig NS yourdomain.xyz +short
curl -sI https://yourdomain.xyz | head -20
```

Which header in the `curl` output proves the request went through Cloudflare?

Alternative (no purchase): Do Part 3 fully, and for Parts 1–2 write a step-by-step plan of exactly what you *would* click, including the two nameserver values Cloudflare would issue and the DNS records you'd expect for apex + `www`. Include the registrar comparison table (prices are public — no purchase needed).

Section 8 — VPS vs VDS (6 pts)

8.1 Define **VPS** and **VDS**. Then explain the real technical difference: which one **oversells/shares** CPU and RAM, and which **dedicates** it? (2 pts)

8.2 Complete the table: (2 pts)

	Shared hosting	VPS	VDS	Dedicated server
Root access				
Resources guaranteed				
Typical monthly cost				
You patch the OS				

8.3 Pick VPS or VDS for each and justify in one line: (1 pt)

- a) A hobby Telegram bot
- b) A production PostgreSQL database with strict latency requirements

8.4 You buy a "4 vCPU" VPS and it feels slower than advertised. Name **two** likely causes and **one command** you'd run to investigate. (1 pt)

Section 9 — Git (8 pts)

9.1 Write the exact command for each: (3 pts)

Task	Command
a	Create a new branch and switch to it
b	Stage only part of a file
c	See the last 5 commits, one line each
d	Undo the last commit but keep the changes staged
e	Discard all uncommitted changes to a tracked file
f	See what changed between your branch and <code>main</code>

9.2 Explain the difference between `git fetch` and `git pull` in one sentence. (1 pt)

9.3 `merge` vs `rebase`: one advantage of each, and one rule about when **not** to rebase. (2 pts)

9.4 You committed a `.env` file with a real password and pushed it. List the steps you take, **in order**. (1 pt)

9.5 Write a good and a bad commit message for the same change, and say what makes the good one good. (1 pt)

Section 10 — Three-Tier Application (8 pts)

10.1 Name the three tiers and give a concrete technology for each. (2 pts)

10.2 Draw the architecture as ASCII art or a Mermaid diagram. Include: user → CDN → load balancer → app servers (×2) → database. Mark **where HTTPS terminates** and **which components are stateless**. (3 pts)

your diagram here

10.3 Why is separating the tiers useful? Give one **scaling** reason and one **security** reason. (1 pt)

10.4 Traffic doubles. Which tier do you scale first, and why is the database usually the hardest? (1 pt)

10.5 Map your Cloudflare Pages site from Lab A onto this model. Which tiers exist, and which are missing? (1 pt)

Section 11 — Interview Preparation (8 pts)

11.1 — Scenario, written answer. Answer as if speaking to an interviewer, ~150–200 words: (3 pts)

"A user reports our website is down. You open it and it loads fine for you. Walk me through how you investigate."

Your answer must state your assumptions out loud and mention at least one tool.

11.2 — Scenario. (2 pts)

"You deployed 20 minutes ago. Error rate went from 0.1% to 4%. What do you do?"

Answer in 5 numbered steps. Say explicitly at which step you'd roll back.

11.3 — Your questions for them. Write **four** questions you would ask the interviewer: one about the team, one about the on-call rotation, one about the tech stack, one about how success is measured in the first 6 months. (2 pts)

11.4 Name one thing you should **never** do in a technical interview when you don't know the answer — and what to do instead. (1 pt)

Grading

Section	Points
1. DevOps: Why?	8
2. Profession	8
3. SRE / SLA / Incidents	14
4. CSR vs SSR	10
5. Cloudflare & Pages	10
6. Lab A — CF Pages deploy	14
7. Lab B — Domain + DNS	16
8. VPS vs VDS	6
9. Git	8
10. Three-tier	8
11. Interview prep	8
Total	110

Pass mark: **70**. Labs A and B are mandatory — a submission missing either is returned unmarked.

Appendix — Useful references

- Cloudflare Pages docs: <https://developers.cloudflare.com/pages/>
- Cloudflare custom domains: <https://developers.cloudflare.com/pages/configuration/custom-domains/>

- Google SRE book, Ch. 4 (SLOs): <https://sre.google/sre-book/service-level-objectives/>
- Uptime / downtime calculator: <https://uptime.is>
- `dig` cheat sheet: `man dig` , or <https://toolbox.googleapps.com/apps/dig/>
- Pro Git book (free): <https://git-scm.com/book/en/v2>